

JUNIOR FULL STACK · TAKE-HOME TASK

EventHub

A small events app. Organizers publish events at venues, and people register to attend. Build it with Node.js, Express, MongoDB and Vue 3.

BACKEND	DATABASE	FRONTEND
Node.js + Express	MongoDB	Vue.js 3

No login in this task — skip authentication entirely. Instead, seed a few users and let the frontend pick “who am I” from a dropdown. That keeps things simple while still connecting users to their events.

01 The data

Four collections. The relationships are the heart of this task.

COLLECTION	FIELDS
User	name, email
Venue	name, city, address, capacity
Event	title, description, startsAt, price, venue, organizer, categories[]
Registration	user, event, ticketCount, createdAt

- A venue has many events, and a user (as organizer) has many events. Store these as **references on the Event document**.
- An event has several categories — a simple array of strings is fine.
- Users and events are many-to-many. Model this with the separate **Registration** collection, *not* with an array inside Event.
- Add a **compound unique index** on Registration (user + event) so the same person cannot register twice for the same event. Enforce it in MongoDB, not only in your JavaScript.
- Deleting an event must not leave its registrations behind.

- An event has **no capacity field of its own**. Its limit is the capacity of the venue hosting it, so you will need to follow the reference to check it.

One endpoint must use an **aggregation pipeline** rather than a simple find: `GET /api/stats/top-venues` — the top 5 venues by number of registrations.

02 The API

Express. Build these routes.

METHOD	ROUTE	DOES
GET	<code>/api/events</code>	List events, with search and filters
GET	<code>/api/events/:id</code>	One event, including venue, organizer and categories in the same response
POST	<code>/api/events</code>	Create an event
PUT	<code>/api/events/:id</code>	Update an event
DELETE	<code>/api/events/:id</code>	Delete an event <i>and</i> its registrations
POST	<code>/api/events/:id/register</code>	Register a user for an event
GET	<code>/api/events/:id/attendees</code>	List the users registered for an event
GET	<code>/api/venues</code>	List venues
GET	<code>/api/users</code>	List users (for the dropdown)
GET	<code>/api/stats/top-venues</code>	Top 5 venues by registrations (aggregation)

GET /api/events should accept

- `q` — a text search across title and description
- Filters for city and category
- Pagination (page + size)

How you implement the text search in MongoDB is up to you. Just explain your choice in `NOTES.md`.

Also

- Validate incoming data and return proper status codes — **400** for bad input, **404** for missing, **409** for a duplicate registration.
- Refuse a registration when the event has reached the capacity of its venue.

- Keep connection strings in environment variables, not in the code.

03 The frontend

Three pages, using Vue 3 and Vue Router.

1. **Events list** — a search box plus city and category filters.
2. **Event detail** — full event info, venue, organizer, the attendee list, and a register button.
3. **Create event** — a form to add an event, picking a venue and an organizer from dropdowns.

Plus a “logged in as” user selector somewhere in the layout.

There is no endpoint listing the available categories. Deriving them from the existing events or hardcoding a short list are both fine — just be consistent.

Handle loading and error states, and show a friendly message when there are no results. **Styling is not part of the evaluation** — plain CSS or any UI library is fine.

04 Extra credit — optional

Only once the parts above work. None of these are expected — any single one is impressive.

- **Elasticsearch** — move the search out of MongoDB and into Elasticsearch. Create an index for events with an explicit mapping, keep it updated when events are created, edited or deleted, and serve the search endpoint from it while MongoDB stays the source of truth. Further bonus: typo tolerance and highlighting the matched words.
- **JWT authentication** — register and login endpoints, hashed passwords, protected routes, and the token sent from Vue. Further bonus: organizer and attendee roles, where an organizer can only edit their own events.
- **A waitlist** instead of rejecting registrations for a full event.
- **docker-compose** that starts the database and the app together — and Elasticsearch too, if you did that part.
- **A few meaningful tests.**

05 How to submit

Reply to our email with a link to a public Git repository (GitHub, GitLab or Bitbucket). Your repo must contain a `README.md` that lets us run the app without asking you anything. Please include:

1. What the app does, in a few lines

2. Requirements and versions (Node, MongoDB)
3. How to start MongoDB
4. The environment variables needed, plus a committed `.env.example`
5. How to install and run the backend and the frontend, with the ports
6. How to run the seed script — please include one that adds a few users, venues and events
7. The list of API endpoints
8. What you completed, what you skipped, and anything known to be broken. Being upfront here counts in your favour and never against you

ALSO ADD A SHORT NOTES.MD — HALF A PAGE IS PLENTY

- Why you stored each relationship the way you did
- How you implemented the text search and why
- What you would improve with more time

Please commit as you go rather than pushing everything in one final commit — we like seeing how a solution develops.

06 Timeline

- Reply to our email to confirm you have read this task, **within 2 days**.
- You then have **5 days from your confirmation** to submit. We will send you the exact date once we hear from you.
- You do not have to finish everything. A partial submission is completely fine, and we would much rather receive one than nothing at all.
- Need more time, or something came up? Just reply and ask. Asking will not count against you.

What happens next. We review every submission we receive. If your work looks good, we invite you to a technical interview where we go through your code together and talk about the decisions you made. There is no whiteboard round and no algorithm puzzles.

07 Tips

- Build in this order: **models** → **API** → **Vue**. Get one feature working end to end before starting the next.
- The fastest way to run MongoDB is Docker. It will save you an afternoon of installation problems.

- Write the seed script early. Having real data to look at makes everything else easier.
- Test your endpoints with Postman before you touch the frontend.
- Do not skip the README. An app we cannot start scores lower than a smaller app we can run in two minutes.
- If you run short on time, finish fewer features properly rather than leaving many half-working. Then write down what you left out.
- If something is unclear, make a decision, note it in NOTES.md, and keep going. Or just email us — questions are genuinely welcome, and a good one is a positive signal.
- We know this stack may be new to you, and that is fine. We are not testing prior Node.js experience — we want to see how you pick up an unfamiliar stack and how you reason about your own code.
- Use whatever tools you normally use, AI assistants included. Just make sure you understand every line, because we will go through the code together in the interview.